TamboUI's architecture follows ratatui closely. If you've used ratatui before, the concepts here will be familiar - `Buffer`, `Cell`, `Frame`, `Widget`, `Layout` all work the same way.

Rendering Model

TamboUI uses **immediate-mode rendering** with an **intermediate buffer system**:

1. **Widgets render to a Buffer** - not directly to the terminal
2. **Buffer is diffed** - only changed cells are sent to the terminal
3. **Each frame is a full redraw** - simple state management, no retained state in widgets

```
Application State
       │
       ▼
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│   Widgets    │ ──►  │   Buffer     │ ──►  │  Terminal    │
│   render()   │      │  (diff)      │      │  (output)    │
└──────────────┘      └──────────────┘      └──────────────┘
```

This model makes TamboUI applications easy to reason about: the UI is always a pure function of your application state.

Buffer System

> If you're using the Toolkit DSL, you won't interact with buffers directly. Feel free to skip this section.

Buffer

A `Buffer` is a 2D grid of `Cell` objects representing the terminal screen:

```
// A Buffer is created for each frame
Buffer buffer = Buffer.empty(new Rect(0, 0, width, height));

// Set a cell at position (x, y)
buffer.set(x, y, new Cell("A", Style.EMPTY.fg(Color.RED)));

// Get a cell
Cell cell = buffer.get(x, y);
```

Cell

A `Cell` represents a single character with styling:

```
// Create a cell with character and style
Cell cell = new Cell("X", Style.EMPTY.bold().fg(Color.CYAN));

// Get properties
String symbol = cell.symbol();
Style style = cell.style();
```

Frame

A `Frame` wraps a buffer and provides rendering helpers:

```
terminal.draw(frame -> {
    // Get the renderable area
    Rect area = frame.area();

    // Render a widget to the frame
    frame.renderWidget(myWidget, area);

    // Or render to a sub-area
    Rect subArea = new Rect(0, 0, 40, 10);
    frame.renderWidget(headerWidget, subArea);
});
```

Layout System

Rect

A `Rect` defines a rectangular region with position and size:

```
// Create a rectangle
Rect rect = new Rect(x, y, width, height);

// Properties
int rx = rect.x();
int ry = rect.y();
int rw = rect.width();
int rh = rect.height();

// Derived values
int right = rect.right();    // x + width
int bottom = rect.bottom();  // y + height
```

Constraints

Constraints define how space is allocated in layouts:

Constraint	Description	Example
`Length(n)`	Fixed size of n cells	`Constraint.length(20)`
`Percentage(n)`	n% of available space	`Constraint.percentage(50)`
`Ratio(num, denom)`	Fractional size	`Constraint.ratio(1, 3)`
`Min(n)`	At least n cells	`Constraint.min(10)`
`Max(n)`	At most n cells	`Constraint.max(50)`

Constraint	Description	Example
`Fill(weight)`	Fill remaining space with weight	`Constraint.fill()` or `Constraint.fill(2)`

Layout

A `Layout` divides a rectangular area into smaller areas based on constraints.

The core operation is `split()`: it takes a `Rect` and returns a `List<Rect>` - one for each constraint. You then render widgets into those rectangles.

```
Layout layout = Layout.vertical()
    .constraints(
        Constraint.length(3),   // First area: 3 cells tall
        Constraint.fill()       // Second area: takes remaining space
    );

// Split a 80x24 area into two rectangles
Rect area = new Rect(0, 0, 80, 24);
List<Rect> areas = layout.split(area);

// Result:
// areas.get(0) = Rect(0, 0, 80, 3)   - header area
// areas.get(1) = Rect(0, 3, 80, 21)  - content area
```

Complete example with rendering:

```
// Split vertically into header (3 lines) and content (rest)
Layout layout = Layout.vertical()
    .constraints(
        Constraint.length(3),
        Constraint.fill()
    );

List<Rect> areas = layout.split(frame.area());
Rect headerArea = areas.get(0);
Rect contentArea = areas.get(1);

// Render widgets into the split areas
```

```
Paragraph header = Paragraph.from("Header Text");
header.render(headerArea, frame.buffer());

Paragraph content = Paragraph.from("Main content goes here...");
content.render(contentArea, frame.buffer());
```

Layouts can be nested:

```
// Split horizontally with percentages
Layout horizontalLayout = Layout.horizontal()
    .constraints(
        Constraint.percentage(30),
        Constraint.percentage(70)
    );

List<Rect> columns = horizontalLayout.split(contentArea);

// Render sidebar and main area
ListWidget sidebar = ListWidget.builder()
    .items("Item 1", "Item 2", "Item 3")
    .build();
sidebar.render(columns.get(0), frame.buffer(), new ListState());

Paragraph mainArea = Paragraph.from("Main content");
mainArea.render(columns.get(1), frame.buffer());
```

Flex Positioning

When layout children don't fill the entire container (e.g., fixed-size elements in a large space), you can control how remaining space is distributed using `Flex`:

```
// Center three fixed-width buttons in a toolbar
Layout toolbar = Layout.horizontal()
    .constraints(
        Constraint.length(10),  // Button 1
        Constraint.length(10),  // Button 2
        Constraint.length(10)   // Button 3
    )
    .flex(Flex.CENTER);  // Center the buttons, space on both sides

List<Rect> buttonAreas = toolbar.split(toolbarArea);
```

```
renderButton("Save", buttonAreas.get(0), frame.buffer());
renderButton("Cancel", buttonAreas.get(1), frame.buffer());
renderButton("Help", buttonAreas.get(2), frame.buffer());

// Spread menu items across a navigation bar
Layout navbar = Layout.horizontal()
    .constraints(
        Constraint.length(8),   // "File"
        Constraint.length(8)    // "Edit"
    )
    .flex(Flex.SPACE_BETWEEN);  // Push to edges with gap between

List<Rect> menuAreas = navbar.split(navbarArea);
renderMenuItem("File", menuAreas.get(0), frame.buffer());
renderMenuItem("Edit", menuAreas.get(1), frame.buffer());
```

Available flex modes: `START`, `CENTER`, `END`, `SPACE_BETWEEN`, `SPACE_AROUND`, `SPACE_EVENLY`.

See Flex Layout in the CSS Styling guide for detailed documentation including CSS usage and practical examples.

Direction

Layouts can split in two directions:

- `Direction.VERTICAL` - stack elements top to bottom
- `Direction.HORIZONTAL` - place elements left to right

Margin

Add spacing around layouts:

```
Layout.vertical()
    .margin(new Margin(1, 2, 1, 2))  // top, right, bottom, left
    .constraints(Constraint.fill());
```

Styling

Style

`Style` is an immutable object that defines text appearance:

```
// Create a style
Style style = Style.EMPTY
    .fg(Color.CYAN)
    .bg(Color.BLACK)
    .bold()
    .underlined();

// Styles are immutable - methods return new instances
Style dimStyle = style.dim();
```

Color

Colors can be specified in multiple ways:

```
// Named ANSI colors
Color red = Color.RED;
Color green = Color.GREEN;
Color cyan = Color.CYAN;
Color white = Color.WHITE;
Color gray = Color.GRAY;

// Indexed colors (0-255)
Color indexed = Color.indexed(196);

// RGB colors (true color)
Color rgb = Color.rgb(255, 128, 0);
```

Modifiers

Text modifiers change how text is displayed:

```
Style.EMPTY
    .bold()       // Bold text
    .dim()        // Dimmed/faint
    .italic()     // Italic (terminal support varies)
```

```
    .underlined() // Underlined
    .slowBlink()  // Slow blinking
    .rapidBlink() // Rapid blinking
    .reversed()   // Swap fg/bg colors
    .hidden()     // Hidden text
    .crossedOut(); // Strikethrough
```

Text System

TamboUI provides a hierarchical text model (`Text` > `Line` > `Span`) for styled terminal output. For a convenient way to create styled text from markup strings, see Markup Text.

Text

`Text` represents multi-line styled text:

```
// Simple text
Text text = Text.from("Hello, World!");

// Multi-line text
Text multiLine = Text.from(
    Line.from("First line"),
    Line.from("Second line")
);

// With alignment
Text centered = Text.from("Centered").alignment(Alignment.CENTER);
```

Line

A `Line` is a single line composed of `Span` objects:

```
Line line = Line.from(
    Span.styled("Bold", Style.EMPTY.bold()),
    Span.raw(" and "),
```

```
    Span.styled("Red", Style.EMPTY.fg(Color.RED))
);
```

Span

A `Span` is a styled piece of text:

```
// Unstyled span
Span plain = Span.raw("plain text");

// Styled span
Span styled = Span.styled("styled", Style.EMPTY.fg(Color.CYAN).bold());
```

Widget Interfaces

Widget (Stateless)

Stateless widgets implement the `Widget` interface:

```
public interface WidgetInterface {
    void render(Rect area, Buffer buffer);
}
```

Examples: `Paragraph`, `Gauge`, `Block`, `Clear`

StatefulWidget (Stateful)

Stateful widgets carry external state:

```
public interface StatefulWidgetInterface<S> {
    void render(Rect area, Buffer buffer, S state);
}
```

The state object tracks selection, scroll position, etc:

```
// Create state
ListState listState = new ListState();

// Render with state
listWidget.render(area, buffer, listState);

// Modify state based on user input
listState.selectNext(items.size());
```

Examples: `ListWidget` , `Table` , `Tabs` , `TextInput`

Event System

TamboUI supports keyboard, mouse, and tick events:

KeyEvent

```
// Check specific keys
if (event.code() == KeyCode.ENTER) { /* handle enter */ }
if (event.code() == KeyCode.CHAR && event.character() == 'q') { /* handle q */ }

// Check modifiers
if (event.modifiers().ctrl()) { /* Ctrl is held */ }

// Using semantic actions
if (event.isQuit()) { /* quit action */ }
if (event.isUp()) { /* move up action */ }
```

MouseEvent

```
int x = event.x();
int y = event.y();
MouseEventKind kind = event.kind();
```

```
if (kind == MouseEventKind.PRESS && event.button() == MouseButton.LEFT) {
    handleClick(x, y);
}
```

TickEvent

Tick events fire periodically for animations:

```
// Configure tick rate
TuiConfig config = TuiConfig.builder()
    .tickRate(Duration.ofMillis(16))  // ~60fps
    .build();

// Handle tick
if (event instanceof TickEvent) {
    updateAnimation();
    // return true to trigger redraw
}
```

Exceptions

TamboUI tries to enforce a clear, consistent exception hierarchy for framework errors.

- All framework exceptions extend `TamboUIException` (`dev.tamboui.error.TamboUIException`). This is a `RuntimeException` .
- Terminal I/O errors are wrapped in `RuntimeIOException` (`dev.tamboui.error.RuntimeIOException`), to avoid having to catch `IOException` which often are non-recoverable.
- Backend (non-I/O) errors use `BackendException` (`dev.tamboui.terminal.BackendException`).
- TUI framework errors use `TuiException` (`dev.tamboui.tui.TuiException`).
- For invalid parameters or state, Java standard exceptions such as `IllegalArgumentException` or `IllegalStateException` are used.
- Domain-specific exception types (e.g., `SolverException` for layout, `CssParseException` for CSS parsing) are used where possible.

TamboUI's `TuiRunner` also provides centralized error handling: all exceptions during rendering and event handling are passed to a configurable `RenderErrorHandler`. The default behavior displays errors (with stack traces) in the UI for easier debugging.

Next Steps

- Explore the Widgets Reference for available components
- Learn about API Levels in detail
- Use CSS Styling for external style definitions
- Understand Bindings and Actions for input handling
- Build maintainable apps with Application Structure

PREVIOUS
Getting Started

NEXT
Widgets Reference